

ĐẠI HỌC QUỐC GIA TP HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN MẠNG MÁY TÍNH VÀ VIỄN THÔNG

MINI VAULT
ASSIGNMENT 1

Sinh viên:

Lê Nhật Duy (23127177)

Nguyễn Minh Kha (23127200)

Giảng viên:

Ths.Lê Giang Thanh

CN.Phan Quốc Kỳ

Ngày 31 tháng 7 năm 2026



Mục lục

1	Thông tin nhóm	1
2	Phân công công việc	1
3	Kiến trúc hệ thống	1
4	Giải thích kỹ thuật và kết quả thực nghiệm	2
4.1	Chức năng 0.1: Khởi tạo và mở khóa Vault	2
4.1.1	Giải thích kỹ thuật	2
4.1.2	Luồng xử lý	3
4.1.3	Kết quả thực nghiệm	3
4.2	Chức năng 0.2: Đăng ký và xác thực người dùng	6
4.2.1	Giải thích kỹ thuật	6
4.2.2	Luồng xử lý	7
4.2.3	Kết quả thực nghiệm	7
4.3	Chức năng 1.1: Lưu trữ KV được mã hóa	8
4.3.1	Giải thích kỹ thuật	8
4.3.2	Luồng xử lý	9
4.3.3	Kết quả thực nghiệm	9
4.4	Chức năng 1.2: Kiểm soát truy cập KV theo quyền sở hữu	11
4.4.1	Giải thích kỹ thuật	11
4.4.2	Luồng xử lý	11
4.4.3	Kết quả thực nghiệm	12
4.5	Chức năng 2.1: Quản lý khóa định danh	13
4.5.1	Giải thích kỹ thuật	13
4.5.2	Luồng xử lý	14
4.5.3	Kết quả thực nghiệm	14
4.6	Chức năng 2.2: Mã hóa và giải mã bằng Transit	16
4.6.1	Giải thích kỹ thuật	16
4.6.2	Luồng xử lý	16
4.6.3	Kết quả thực nghiệm	17

4.7	Chức năng 2.3: Kiểm soát truy cập khóa định danh	18
4.7.1	Giải thích kỹ thuật	18
4.7.2	Luồng xử lý	19
4.7.3	Kết quả thực nghiệm	19
4.8	Chức năng 2.4: Ký và xác minh chữ ký	20
4.8.1	Giải thích kỹ thuật	20
4.8.2	Luồng xử lý	21
4.8.3	Kết quả thực nghiệm	21
5	Kiểm tra dữ liệu mẫu	23
5.1	Cơ sở dữ liệu KV được mã hóa	23
5.2	Transit ciphertext mẫu	24
6	Các chức năng nâng cao	25
6.1	Xác thực đa yếu tố bằng TOTP	26
6.1.1	Giải thích kỹ thuật	26
6.1.2	Kết quả thực nghiệm	26
6.2	Audit log chống chỉnh sửa	26
6.2.1	Giải thích kỹ thuật	26
6.2.2	Kết quả thực nghiệm	26
6.3	Quản lý phiên bản KV	27
6.3.1	Giải thích kỹ thuật	27
6.3.2	Kết quả thực nghiệm	27
6.4	Xoay vòng khóa Transit	27
6.4.1	Giải thích kỹ thuật	27
6.4.2	Kết quả thực nghiệm	28
6.5	Phân quyền bằng ACL	28
6.5.1	Giải thích kỹ thuật	28
6.5.2	Kết quả thực nghiệm	28
6.6	Chia sẻ quyền xác minh chữ ký	29
6.6.1	Giải thích kỹ thuật	29
6.6.2	Kết quả thực nghiệm	29

6.7	Shamir's Secret Sharing	30
6.7.1	Giải thích kỹ thuật	30
6.7.2	Kết quả thực nghiệm	30

1 Thông tin nhóm

STT	Họ và tên	MSSV
1	Lê Nhật Duy	23127177
2	Nguyễn Minh Kha	23127200

Bảng 1: Danh sách sinh viên

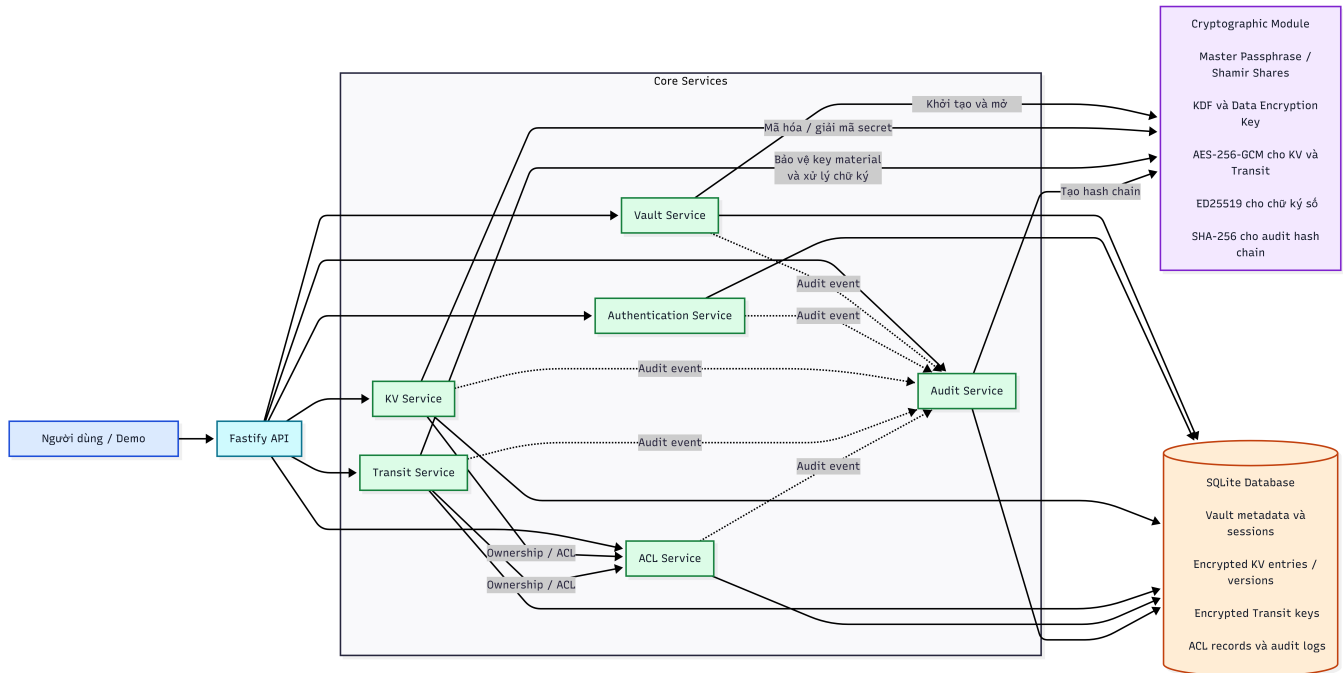
2 Phân công công việc

Sinh viên	Nội dung thực hiện
Sinh viên 1	Thực hiện các chức năng 0.1, 0.2, 2.1 và 2.4; tích hợp các thành phần của hệ thống; xây dựng các kiểm thử liên quan; triển khai và kiểm thử các chức năng nâng cao.
Sinh viên 2	Thực hiện các chức năng 1.1, 1.2, 2.2 và 2.3; xây dựng kiểm thử phân quyền; thực hiện video demo; hỗ trợ kiểm thử các chức năng nâng cao và hoàn thiện phần báo cáo tương ứng.

Bảng 2: Phân công công việc

3 Kiến trúc hệ thống

Mini Vault được xây dựng theo kiến trúc phân lớp, gồm Fastify API, các dịch vụ nghiệp vụ, module mật mã và cơ sở dữ liệu SQLite. Request được chuyển đến dịch vụ tương ứng để xử lý xác thực, phân quyền, mã hóa, quản lý KV và Transit, đồng thời ghi nhận các sự kiện quan trọng vào audit log.



Hình 1: Kiến trúc tổng thể của hệ thống Mini Vault

4 Giải thích kỹ thuật và kết quả thực nghiệm

Các chức năng bắt buộc của Mini Vault được kiểm tra bằng script `scripts/demo.ts`, thông qua các request gửi đến server Fastify chạy trong môi trường cục bộ. Kết quả được đối chiếu bằng HTTP status, nội dung response và các giá trị kiểm tra, trong khi session token và key material không được hiển thị đầy đủ.

4.1 Chức năng 0.1: Khởi tạo và mở khóa Vault

4.1.1 Giải thích kỹ thuật

Vault là thành phần quản lý khóa mã hóa dữ liệu của hệ thống. Trước khi các chức năng KV và Transit có thể hoạt động, Vault phải được khởi tạo và chuyển sang trạng thái **UNLOCKED**.

Quá trình khởi tạo được thực hiện bằng lệnh:

```
npm run vault:init
```

Người dùng cung cấp một *Master Passphrase*. Giá trị này không được lưu trực tiếp trong cơ sở dữ liệu mà được sử dụng cùng salt ngẫu nhiên và Argon2id để dẫn xuất *Key Encryption Key* (KEK).

Hệ thống tiếp tục sinh một *Data Encryption Key* (DEK) ngẫu nhiên có độ dài 256 bit và mã hóa DEK bằng KEK thông qua AES-GCM. Các thông tin cần thiết cho quá trình mở khóa, gồm salt, tham số KDF, nonce, DEK đã mã hóa và authentication tag, được lưu trong bảng `vault_metadata`. Khi server khởi động, Vault ban đầu ở trạng thái `LOCKED`. Sau khi người dùng nhập đúng Master Passphrase, hệ thống dẫn xuất lại KEK và giải mã DEK. DEK được lưu trong `VaultState` và chỉ tồn tại trong bộ nhớ của tiến trình đang chạy. Nếu mở khóa thất bại, Vault vẫn giữ trạng thái `LOCKED`.

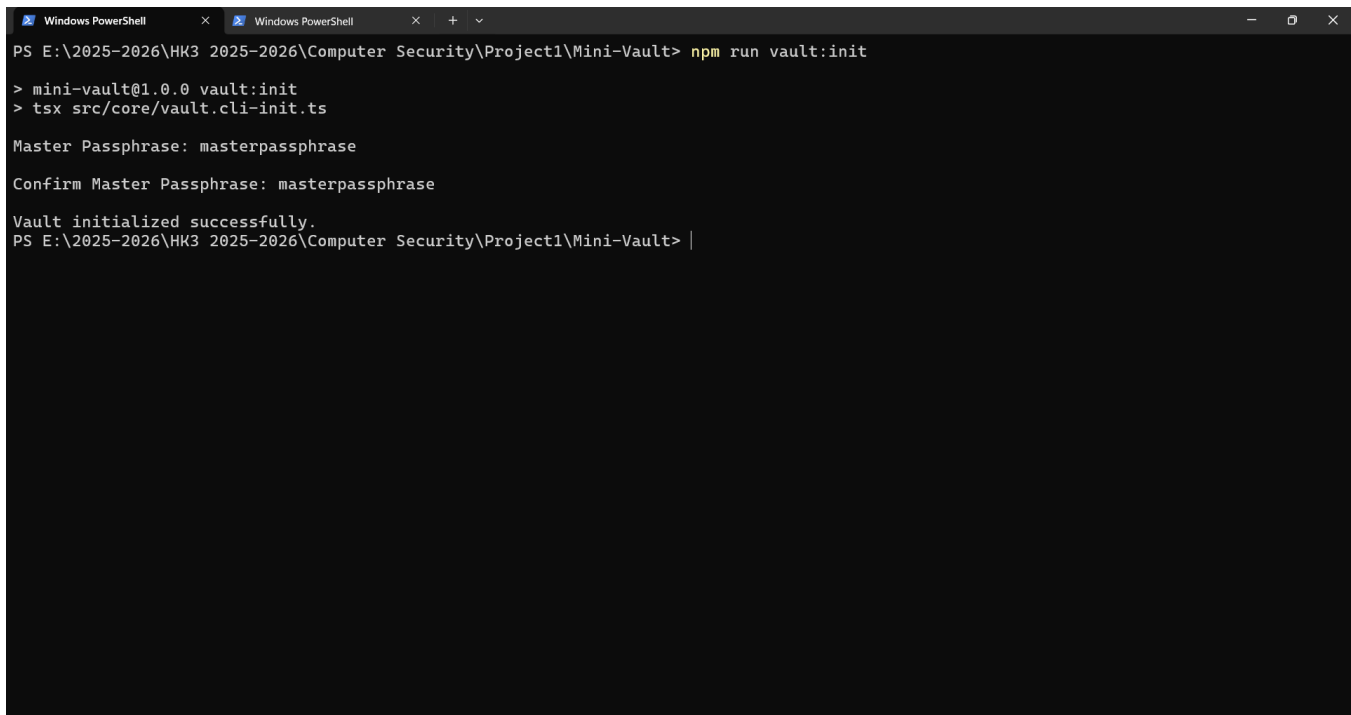
Sau khi sử dụng, các buffer tạm chứa khóa được xóa khỏi bộ nhớ bằng cơ chế zeroization.

4.1.2 Luồng xử lý

1. Người dùng chạy `npm run vault:init` và cung cấp Master Passphrase.
2. Argon2id dẫn xuất KEK từ Master Passphrase và salt ngẫu nhiên.
3. Hệ thống sinh DEK và mã hóa DEK bằng AES-GCM.
4. Metadata cùng DEK đã mã hóa được lưu vào SQLite.
5. Khi server khởi động, Vault ở trạng thái `LOCKED`.
6. Người dùng nhập Master Passphrase để mở khóa.
7. Nếu giải mã DEK thành công, Vault chuyển sang trạng thái `UNLOCKED`.

4.1.3 Kết quả thực nghiệm

Vault được khởi tạo thành công bằng Master Passphrase thông qua lệnh `npm run vault:init`.



```
PS E:\2025-2026\HK3 2025-2026\Computer Security\Project1\Mini-Vault> npm run vault:init

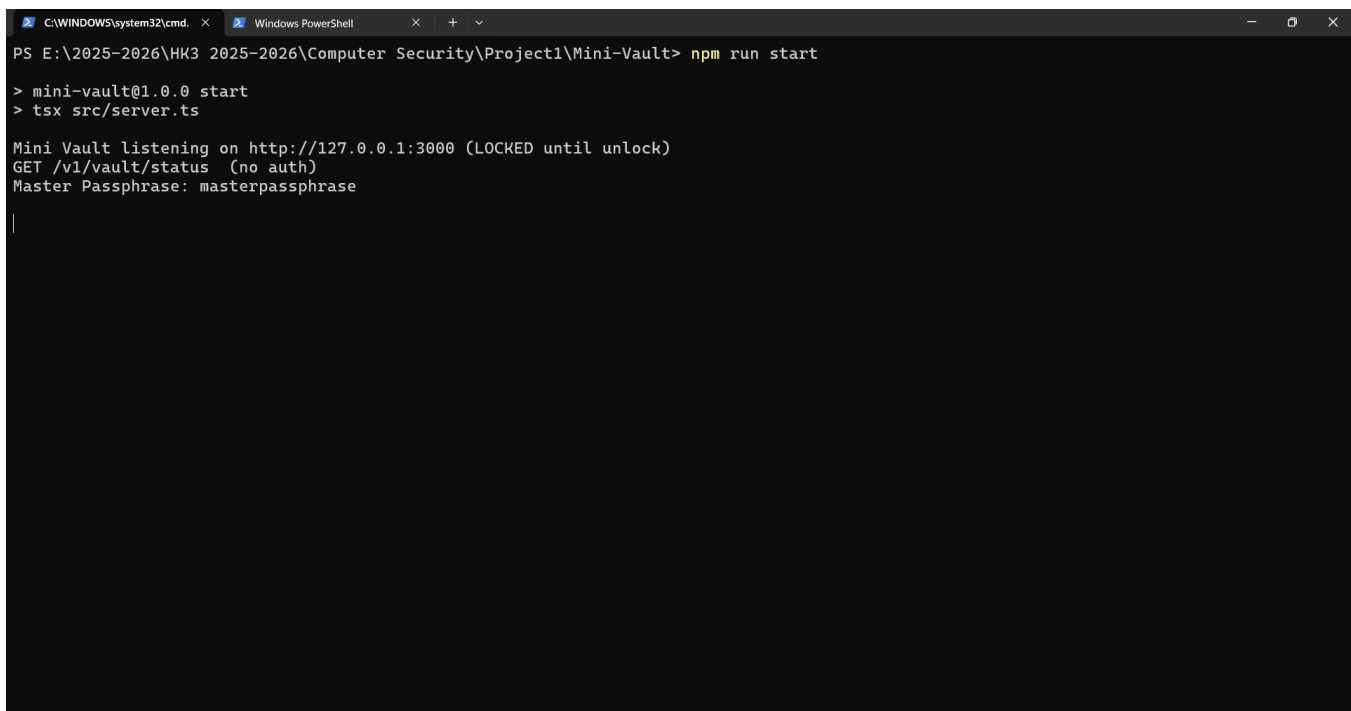
> mini-vault@1.0.0 vault:init
> tsx src/core/vault.cli-init.ts

Master Passphrase: masterpassphrase
Confirm Master Passphrase: masterpassphrase

Vault initialized successfully.
PS E:\2025-2026\HK3 2025-2026\Computer Security\Project1\Mini-Vault> |
```

Hình 2: Khởi tạo Vault bằng Master Passphrase

Sau khi chạy `npm run start`, server bắt đầu lắng nghe tại `http://127.0.0.1:3000`. Trước khi Master Passphrase được xác minh thành công, Vault vẫn ở trạng thái LOCKED.



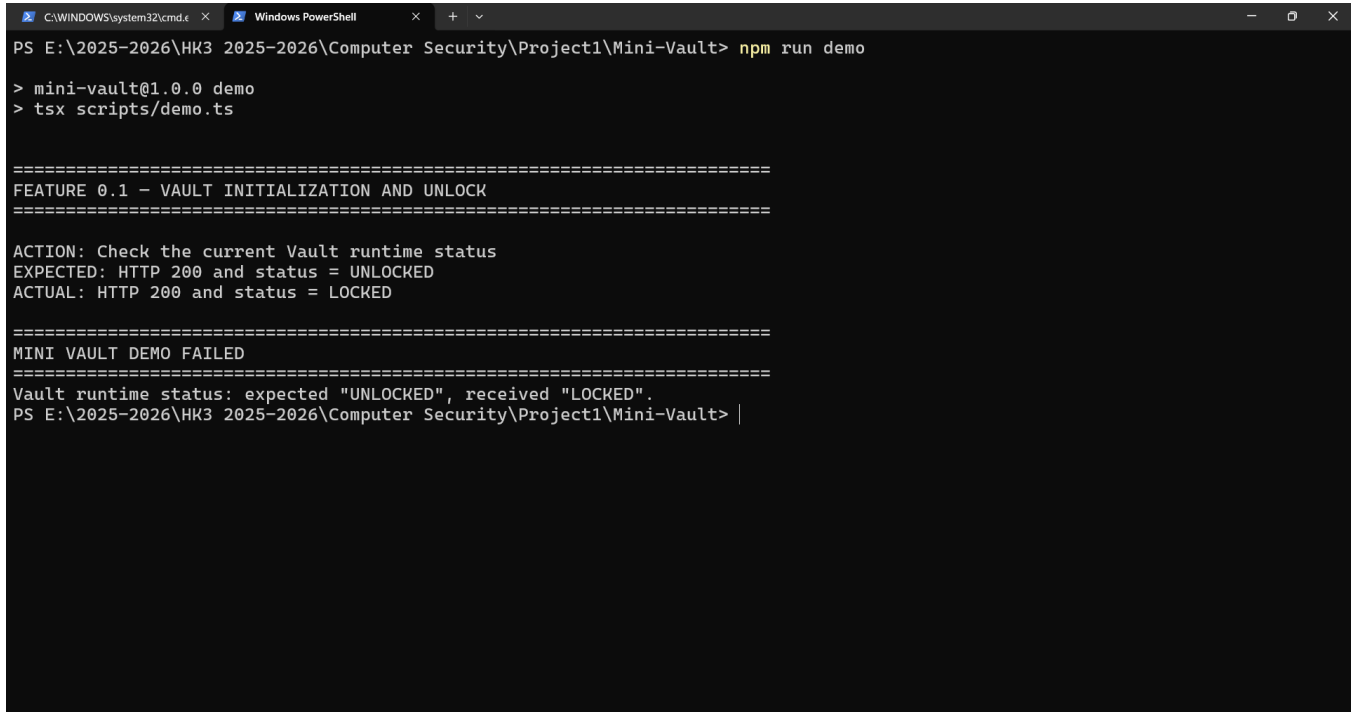
```
PS E:\2025-2026\HK3 2025-2026\Computer Security\Project1\Mini-Vault> npm run start

> mini-vault@1.0.0 start
> tsx src/server.ts

Mini Vault listening on http://127.0.0.1:3000 (LOCKED until unlock)
GET /v1/vault/status (no auth)
Master Passphrase: masterpassphrase
|
```

Hình 3: Khởi động server khi Vault đang ở trạng thái LOCKED

Khi chạy demo trong lúc Vault chưa được mở khóa, endpoint trạng thái vẫn trả về HTTP 200, nhưng trạng thái thực tế là LOCKED. Script demo phát hiện điều kiện tiên đề chưa đạt và dừng trước khi thực hiện các chức năng KV và Transit.



```
C:\WINDOWS\system32\cmd.exe x Windows PowerShell x + v
PS E:\2025-2026\HK3 2025-2026\Computer Security\Project1\Mini-Vault> npm run demo

> mini-vault@1.0.0 demo
> tsx scripts/demo.ts

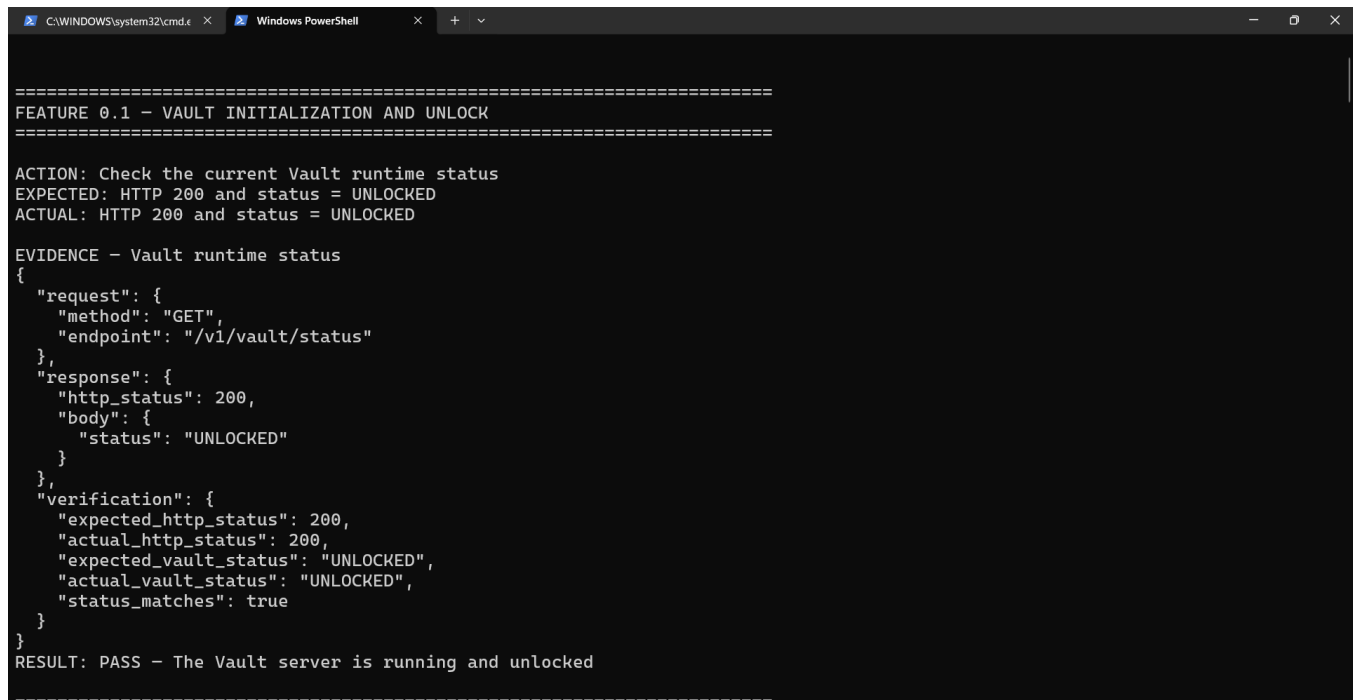
=====
FEATURE 0.1 - VAULT INITIALIZATION AND UNLOCK
=====

ACTION: Check the current Vault runtime status
EXPECTED: HTTP 200 and status = UNLOCKED
ACTUAL: HTTP 200 and status = LOCKED

=====
MINI VAULT DEMO FAILED
=====
Vault runtime status: expected "UNLOCKED", received "LOCKED".
PS E:\2025-2026\HK3 2025-2026\Computer Security\Project1\Mini-Vault> |
```

Hình 4: Script demo dừng khi Vault chưa được mở khóa

Sau khi Master Passphrase chính xác được cung cấp, request `GET /v1/vault/status` trả về HTTP 200 cùng trạng thái UNLOCKED. Giá trị `status_matches = true` xác nhận kết quả thực tế khớp với kết quả mong đợi.



```
=====  
FEATURE 0.1 - VAULT INITIALIZATION AND UNLOCK  
=====
```

```
ACTION: Check the current Vault runtime status  
EXPECTED: HTTP 200 and status = UNLOCKED  
ACTUAL: HTTP 200 and status = UNLOCKED
```

```
EVIDENCE - Vault runtime status  
{  
  "request": {  
    "method": "GET",  
    "endpoint": "/v1/vault/status"  
  },  
  "response": {  
    "http_status": 200,  
    "body": {  
      "status": "UNLOCKED"  
    }  
  },  
  "verification": {  
    "expected_http_status": 200,  
    "actual_http_status": 200,  
    "expected_vault_status": "UNLOCKED",  
    "actual_vault_status": "UNLOCKED",  
    "status_matches": true  
  }  
}
```

```
RESULT: PASS - The Vault server is running and unlocked
```

Hình 5: Xác minh Vault đã chuyển sang trạng thái UNLOCKED

4.2 Chức năng 0.2: Đăng ký và xác thực người dùng

4.2.1 Giải thích kỹ thuật

Chức năng đăng ký và xác thực được triển khai bởi `AuthService` và `AuthRepository`. Thông tin tài khoản được lưu trong bảng `users`, còn các phiên đăng nhập được lưu trong bảng `sessions`.

Khi đăng ký, hệ thống chuẩn hóa email, kiểm tra dữ liệu đầu vào và sử dụng `Argon2id` để băm passphrase. Passphrase gốc không được lưu trực tiếp trong cơ sở dữ liệu.

Khi đăng nhập thành công, hệ thống sinh một session token ngẫu nhiên. Token gốc được trả về cho client, trong khi cơ sở dữ liệu chỉ lưu giá trị SHA-256 của token cùng thông tin người dùng và thời gian hết hạn của phiên.

Trong các request tiếp theo, client gửi token thông qua HTTP header:

```
Authorization: Bearer <session-token>
```

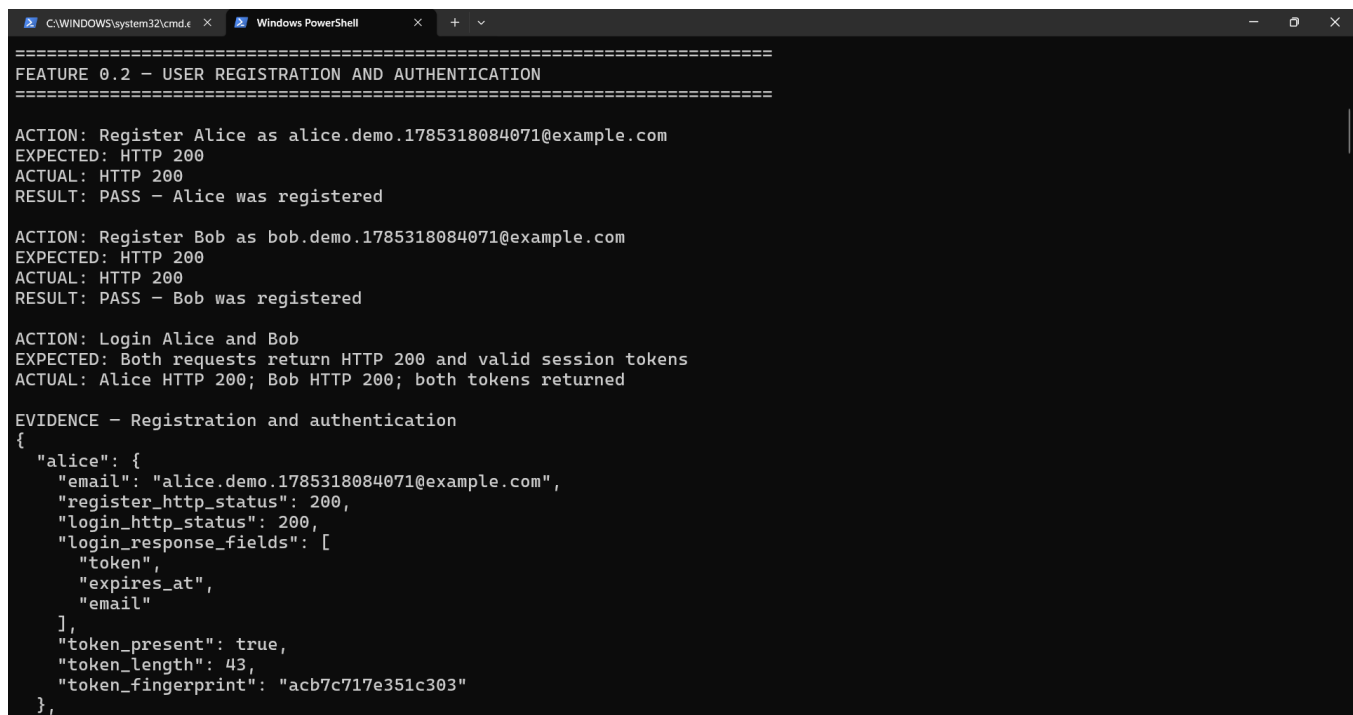
Server băm token nhận được và kiểm tra session tương ứng. Một session chỉ được chấp nhận khi tồn tại, chưa hết hạn và chưa bị thu hồi. Hệ thống cũng hỗ trợ khóa tài khoản tạm thời khi số lần đăng nhập thất bại vượt quá giới hạn cho phép.

4.2.2 Luồng xử lý

1. Alice và Bob gửi thông tin đăng ký đến server.
2. Hệ thống chuẩn hóa email và kiểm tra dữ liệu đầu vào.
3. Passphrase được băm bằng Argon2id và lưu vào bảng `users`.
4. Alice và Bob gửi request đăng nhập.
5. Server xác minh passphrase với password hash đã lưu.
6. Mỗi người dùng được cấp một session token riêng.
7. SHA-256 hash của token được lưu vào bảng `sessions`.
8. Client sử dụng token để xác thực các request tiếp theo.

4.2.3 Kết quả thực nghiệm

Alice và Bob đều được đăng ký và đăng nhập thành công. Các request tương ứng trả về HTTP 200.



```
=====
FEATURE 0.2 - USER REGISTRATION AND AUTHENTICATION
=====

ACTION: Register Alice as alice.demo.1785318084071@example.com
EXPECTED: HTTP 200
ACTUAL: HTTP 200
RESULT: PASS - Alice was registered

ACTION: Register Bob as bob.demo.1785318084071@example.com
EXPECTED: HTTP 200
ACTUAL: HTTP 200
RESULT: PASS - Bob was registered

ACTION: Login Alice and Bob
EXPECTED: Both requests return HTTP 200 and valid session tokens
ACTUAL: Alice HTTP 200; Bob HTTP 200; both tokens returned

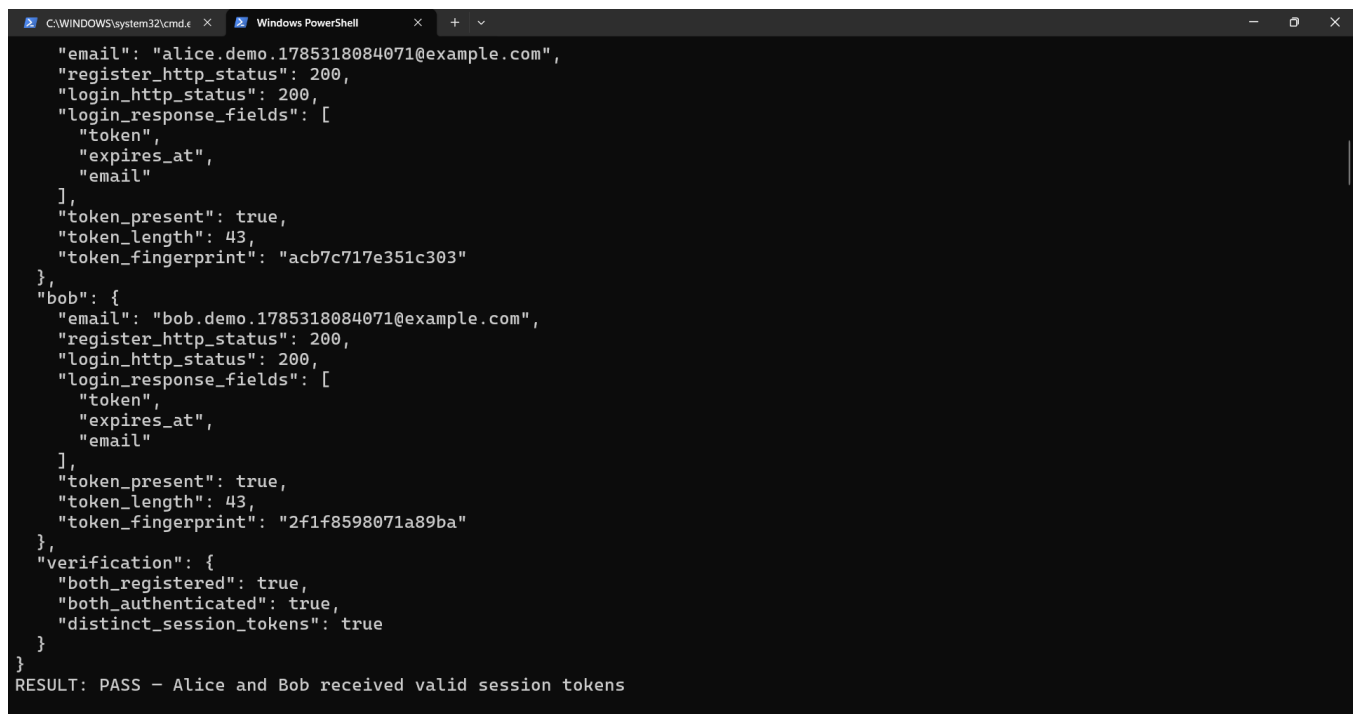
EVIDENCE - Registration and authentication
{
  "alice": {
    "email": "alice.demo.1785318084071@example.com",
    "register_http_status": 200,
    "login_http_status": 200,
    "login_response_fields": [
      "token",
      "expires_at",
      "email"
    ],
    "token_present": true,
    "token_length": 43,
    "token_fingerprint": "acb7c717e351c303"
  },
}
```

Hình 6: Đăng ký và đăng nhập hai người dùng Alice và Bob

Script demo không hiển thị đầy đủ session token mà chỉ cung cấp độ dài và fingerprint rút gọn. Kết quả kiểm tra cho thấy:

- both_registered = true;
- both_authenticated = true;
- token_present = true;
- distinct_session_tokens = true.

Các kết quả trên xác nhận hai người dùng được xác thực thành công và nhận hai session token độc lập.



```
C:\WINDOWS\system32\cmd.exe x Windows PowerShell x + v
{
  "email": "alice.demo.1785318084071@example.com",
  "register_http_status": 200,
  "login_http_status": 200,
  "login_response_fields": [
    "token",
    "expires_at",
    "email"
  ],
  "token_present": true,
  "token_length": 43,
  "token_fingerprint": "acb7c717e351c303"
},
{
  "email": "bob.demo.1785318084071@example.com",
  "register_http_status": 200,
  "login_http_status": 200,
  "login_response_fields": [
    "token",
    "expires_at",
    "email"
  ],
  "token_present": true,
  "token_length": 43,
  "token_fingerprint": "2f1f8598071a89ba"
},
{
  "verification": {
    "both_registered": true,
    "both_authenticated": true,
    "distinct_session_tokens": true
  }
}
RESULT: PASS - Alice and Bob received valid session tokens
```

Hình 7: Xác minh hai người dùng nhận được session token độc lập

Luồng đăng nhập mở rộng bằng TOTP được trình bày trong phần các chức năng nâng cao.

4.3 Chức năng 1.1: Lưu trữ KV được mã hóa

4.3.1 Giải thích kỹ thuật

KV Secret Storage cho phép người dùng lưu trữ dữ liệu JSON theo mô hình key-value. Mỗi secret được định danh bằng một KV path có dạng:

`secret/<owner-email>/<resource>`

Khi ghi secret, dữ liệu JSON được tuần tự hóa và mã hóa bằng AES-GCM với DEK đang được giữ trong `VaultState`. Cơ sở dữ liệu không lưu plaintext mà chỉ lưu KV path, chủ sở hữu, nonce, ciphertext, authentication tag và các thông tin liên quan.

AES-GCM bảo đảm tính bí mật của dữ liệu thông qua ciphertext và kiểm tra tính toàn vẹn thông qua authentication tag. Khi đọc secret, hệ thống sử dụng DEK trong bộ nhớ để giải mã bản ghi và khôi phục đối tượng JSON ban đầu.

Ngoài việc đọc phiên bản hiện tại, hệ thống còn hỗ trợ lưu và truy xuất các phiên bản cũ thông qua chức năng KV versioning, được trình bày trong phần các chức năng nâng cao.

4.3.2 Luồng xử lý

1. Alice gửi KV path và dữ liệu JSON đến server.
2. Server xác thực session và kiểm tra quyền truy cập.
3. Hệ thống xác minh Vault đang ở trạng thái UNLOCKED.
4. Dữ liệu JSON được mã hóa bằng AES-GCM với DEK.
5. Nonce, ciphertext và authentication tag được lưu vào SQLite.
6. Khi đọc, hệ thống lấy bản ghi và giải mã bằng DEK.
7. Dữ liệu JSON ban đầu được trả về cho Alice.

4.3.3 Kết quả thực nghiệm

Alice ghi một JSON secret gồm hai trường `username` và `password` vào namespace của mình. Sau đó, Alice đọc lại secret bằng cùng KV path. Cả hai request đều trả về HTTP 200 và dữ liệu đọc lại có đầy đủ các trường ban đầu.

```
C:\WINDOWS\system32\cmd.exe x Windows PowerShell x + v
=====
FEATURE 1.1 - ENCRYPTED KV STORAGE
=====

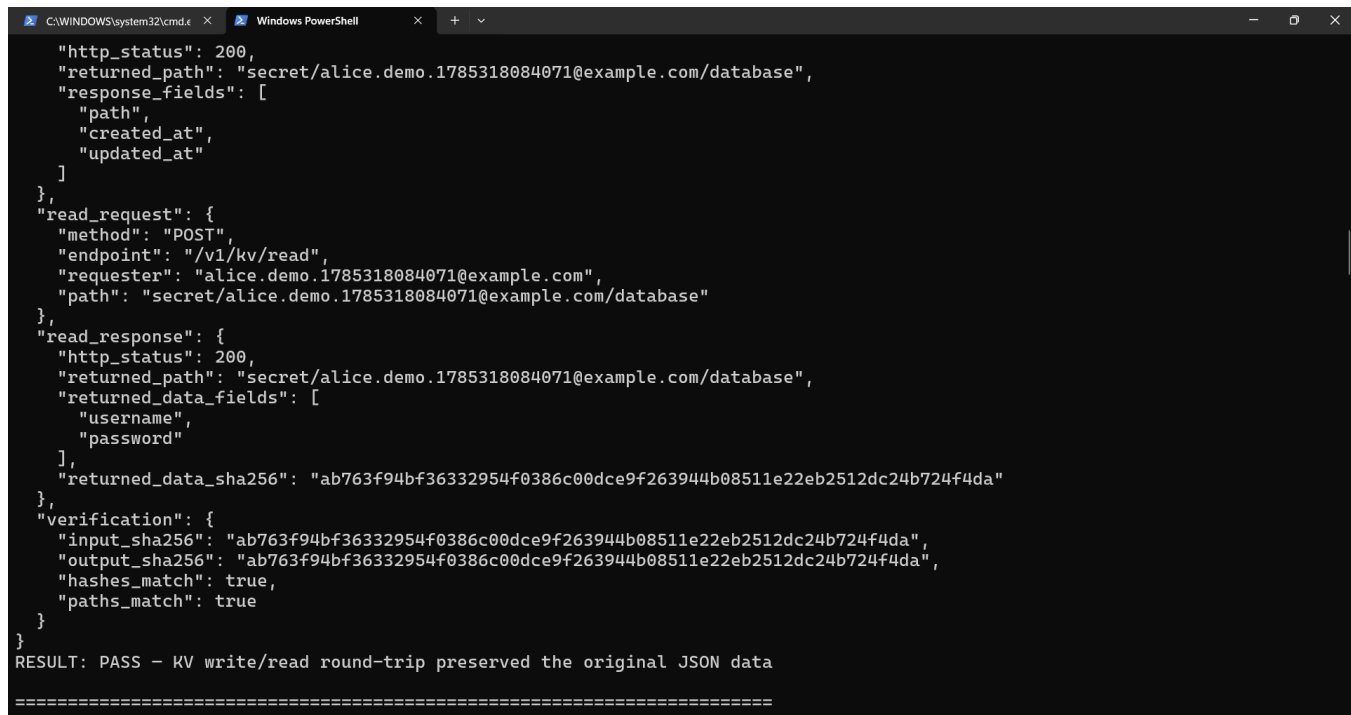
ACTION: Alice writes a JSON secret into her own namespace
EXPECTED: HTTP 200 and path = secret/alice.demo.1785318084071@example.com/database
ACTUAL: HTTP 200 and path = secret/alice.demo.1785318084071@example.com/database
RESULT: PASS - Alice wrote her secret successfully

ACTION: Alice reads the secret from her own namespace
EXPECTED: HTTP 200 and the original JSON data
ACTUAL: HTTP 200;fields = username, password; SHA-256 = ab763f94bf36332954f0386c00dce9f263944b08511e22eb2512dc24b724f4da

EVIDENCE - KV write/read round trip
{
  "write_request": {
    "method": "POST",
    "endpoint": "/v1/kv/write",
    "requester": "alice.demo.1785318084071@example.com",
    "path": "secret/alice.demo.1785318084071@example.com/database",
    "data_fields": [
      "username",
      "password"
    ],
    "input_data_sha256": "ab763f94bf36332954f0386c00dce9f263944b08511e22eb2512dc24b724f4da"
  },
  "write_response": {
    "http_status": 200,
    "returned_path": "secret/alice.demo.1785318084071@example.com/database",
    "response_fields": [
      "path",
      "created_at",
      "updated_at"
    ]
  }
}
```

Hình 8: Alice ghi và đọc lại KV secret trong namespace của mình

Script demo tính SHA-256 của dữ liệu trước khi ghi và sau khi đọc. Kết quả `hashes_match = true` và `paths_match = true`, xác nhận nội dung JSON và KV path được bảo toàn sau quá trình mã hóa và giải mã.



```
"http_status": 200,
"returned_path": "secret/alice.demo.1785318084071@example.com/database",
"response_fields": [
  "path",
  "created_at",
  "updated_at"
],
},
"read_request": {
  "method": "POST",
  "endpoint": "/v1/kv/read",
  "requester": "alice.demo.1785318084071@example.com",
  "path": "secret/alice.demo.1785318084071@example.com/database"
},
"read_response": {
  "http_status": 200,
  "returned_path": "secret/alice.demo.1785318084071@example.com/database",
  "returned_data_fields": [
    "username",
    "password"
  ],
  "returned_data_sha256": "ab763f94bf36332954f0386c00dce9f263944b08511e22eb2512dc24b724f4da"
},
"verification": {
  "input_sha256": "ab763f94bf36332954f0386c00dce9f263944b08511e22eb2512dc24b724f4da",
  "output_sha256": "ab763f94bf36332954f0386c00dce9f263944b08511e22eb2512dc24b724f4da",
  "hashes_match": true,
  "paths_match": true
}
}
RESULT: PASS - KV write/read round-trip preserved the original JSON data
=====
```

Hình 9: Đối chiếu hash của dữ liệu trước khi ghi và sau khi đọc

4.4 Chức năng 1.2: Kiểm soát truy cập KV theo quyền sở hữu

4.4.1 Giải thích kỹ thuật

Theo chính sách mặc định, người dùng chỉ được thao tác với KV secret thuộc namespace của mình. Việc đã đăng nhập thành công không đồng nghĩa với việc người dùng có quyền truy cập dữ liệu của tài khoản khác.

Việc kiểm tra quyền được thực hiện bởi `OwnershipKvAuthorization`. Thành phần này xác định chủ sở hữu từ KV path và so sánh với email của người gửi request được lấy từ session token.

Nếu người dùng không phải chủ sở hữu, hệ thống kiểm tra quyền ACL tương ứng. Khi không có quyền phù hợp, request bị từ chối với HTTP 403 và mã lỗi `PERMISSION_DENIED`.

Quá trình kiểm tra được thực hiện trước khi ciphertext được truy xuất và giải mã. Vì vậy, người dùng không có quyền không thể nhận plaintext hoặc ciphertext của người dùng khác thông qua API. Sự kiện bị từ chối cũng được ghi nhận bởi `AuditService`.

4.4.2 Luồng xử lý

1. Bob gửi request đọc KV kèm session token hợp lệ.

2. Server xác thực token và xác định requester là Bob.
3. Hệ thống xác định Alice là chủ sở hữu của KV path.
4. Quyền sở hữu và ACL của Bob được kiểm tra.
5. Do Bob không có quyền phù hợp, request bị từ chối trước khi dữ liệu được truy xuất và giải mã.
6. Server trả về HTTP 403 và PERMISSION_DENIED.

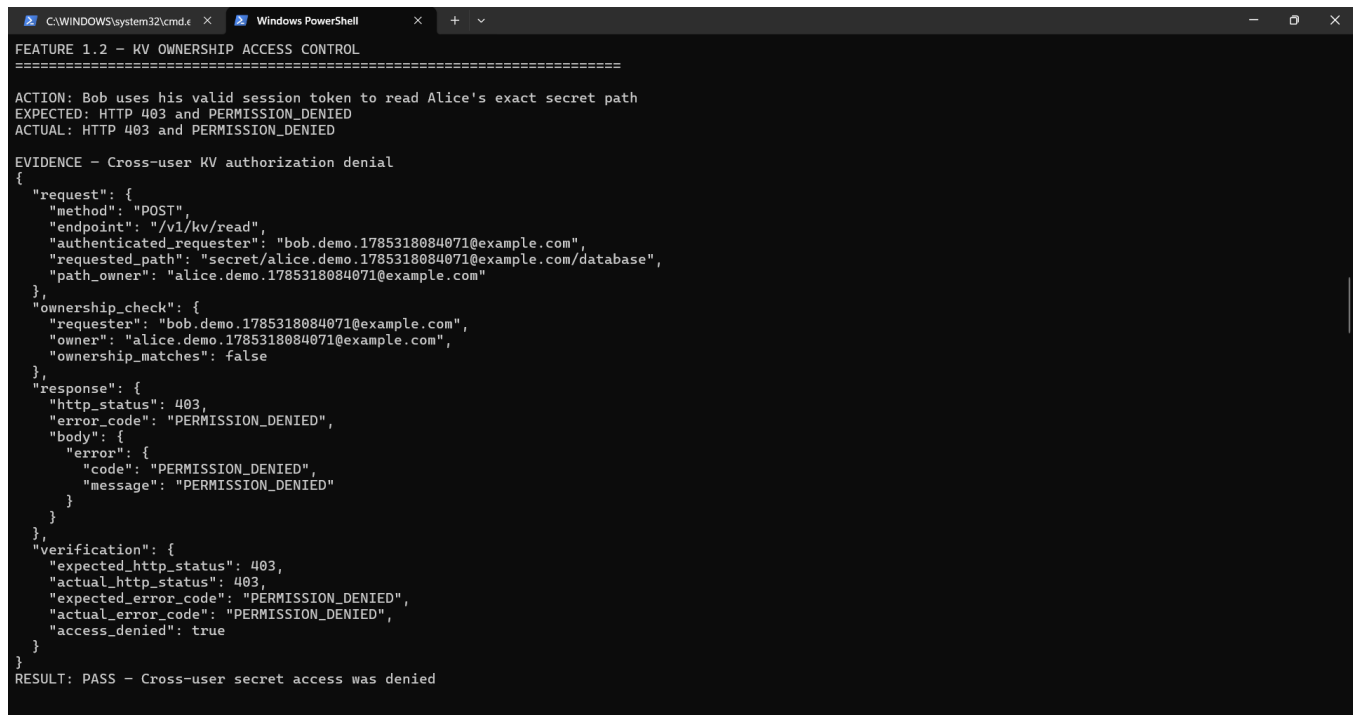
4.4.3 Kết quả thực nghiệm

Bob sử dụng session token hợp lệ để đọc KV secret đã được Alice tạo trong Chức năng 1.1. Tại thời điểm kiểm thử, Alice chưa cấp quyền truy cập secret này cho Bob.

Kết quả cho thấy:

- `ownership_matches = false;`
- HTTP status thực tế là 403;
- mã lỗi là PERMISSION_DENIED;
- `access_denied = true.`

Kết quả xác nhận Bob đã được xác thực nhưng vẫn không thể truy cập KV secret của Alice khi không phải chủ sở hữu và chưa được cấp quyền.



```
FEATURE 1.2 - KV OWNERSHIP ACCESS CONTROL
=====

ACTION: Bob uses his valid session token to read Alice's exact secret path
EXPECTED: HTTP 403 and PERMISSION_DENIED
ACTUAL: HTTP 403 and PERMISSION_DENIED

EVIDENCE - Cross-user KV authorization denial
{
  "request": {
    "method": "POST",
    "endpoint": "/v1/kv/read",
    "authenticated_requester": "bob.demo.1785318084071@example.com",
    "requested_path": "secret/alice.demo.1785318084071@example.com/database",
    "path_owner": "alice.demo.1785318084071@example.com"
  },
  "ownership_check": {
    "requester": "bob.demo.1785318084071@example.com",
    "owner": "alice.demo.1785318084071@example.com",
    "ownership_matches": false
  },
  "response": {
    "http_status": 403,
    "error_code": "PERMISSION_DENIED",
    "body": {
      "error": {
        "code": "PERMISSION_DENIED",
        "message": "PERMISSION_DENIED"
      }
    }
  },
  "verification": {
    "expected_http_status": 403,
    "actual_http_status": 403,
    "expected_error_code": "PERMISSION_DENIED",
    "actual_error_code": "PERMISSION_DENIED",
    "access_denied": true
  }
}
RESULT: PASS - Cross-user secret access was denied
```

Hình 10: Từ chối Bob truy cập KV secret của Alice khi chưa được cấp quyền

4.5 Chức năng 2.1: Quản lý khóa định danh

4.5.1 Giải thích kỹ thuật

Transit Engine cho phép người dùng tạo các khóa có tên định danh, gọi là *named key*. Hệ thống hỗ trợ hai mục đích sử dụng:

- ENCRYPT_DECRYPT: mã hóa và giải mã dữ liệu;
- SIGN_VERIFY: ký và xác minh chữ ký.

Khi tạo khóa ENCRYPT_DECRYPT, hệ thống sinh một khóa AES 256-bit ngẫu nhiên. Key material không được lưu trực tiếp mà được mã hóa bằng DEK đang được giữ trong `VaultState`. Bảng `transit_keys` chỉ lưu tên khóa, chủ sở hữu, mục đích sử dụng, nonce, key material đã mã hóa, authentication tag và metadata liên quan.

API tạo khóa chỉ trả về các thông tin an toàn như `key_name` và `key_usage`; raw key, private key và encrypted key material không xuất hiện trong response. Sau khi hoàn tất, buffer chứa key material gốc được zeroize.

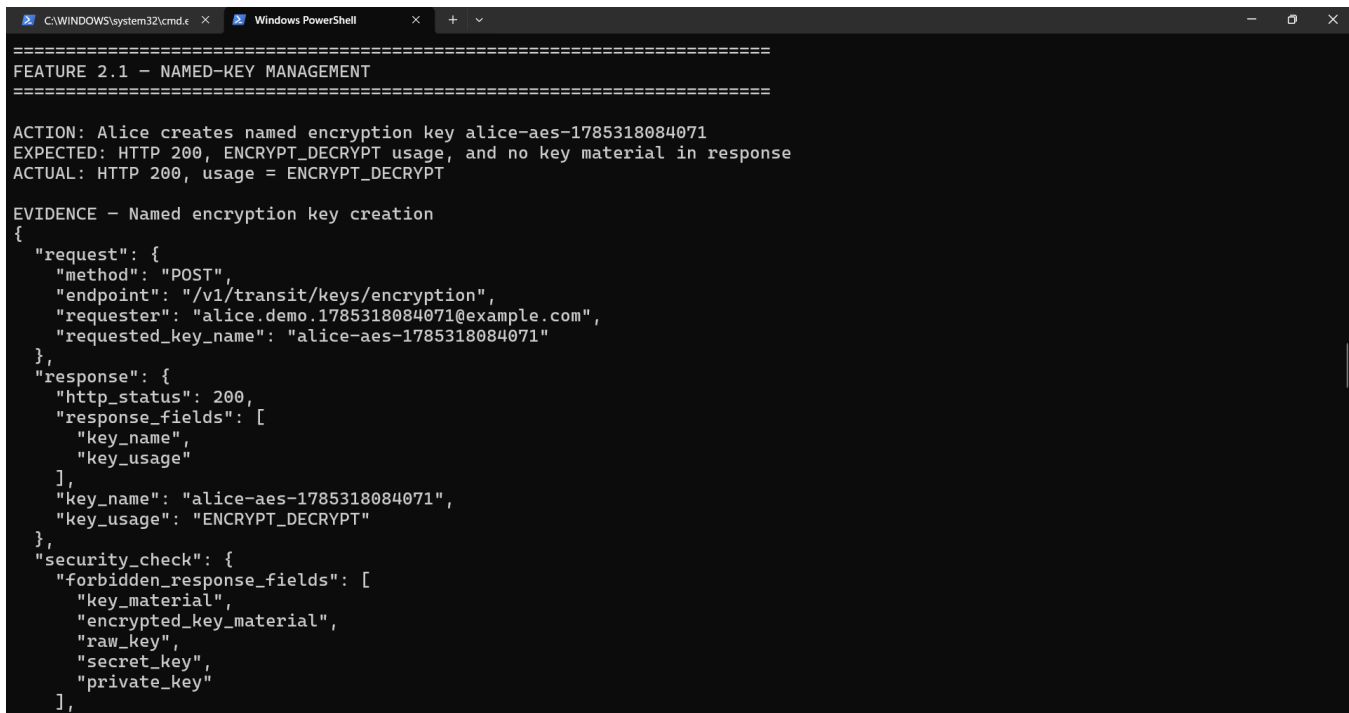
Hệ thống còn hỗ trợ xoay vòng named key và duy trì các phiên bản khóa cũ. Chức năng này được trình bày trong phần Transit key rotation.

4.5.2 Luồng xử lý

1. Alice gửi tên khóa và mục đích sử dụng đến server.
2. Server xác thực session và kiểm tra trạng thái Vault.
3. Hệ thống kiểm tra tính hợp lệ của tên khóa.
4. Một khóa AES 256-bit ngẫu nhiên được sinh ra.
5. Key material được mã hóa bằng DEK thông qua AES-GCM.
6. Bản ghi khóa đã mã hóa được lưu vào cơ sở dữ liệu.
7. API chỉ trả về metadata an toàn của named key.
8. Buffer chứa key material gốc được zeroize.

4.5.3 Kết quả thực nghiệm

Alice tạo một named key có mục đích sử dụng `ENCRYPT_DECRYPT`. Request trả về HTTP 200; tên khóa và mục đích sử dụng trong response khớp với giá trị đã yêu cầu.



```
=====
FEATURE 2.1 - NAMED-KEY MANAGEMENT
=====

ACTION: Alice creates named encryption key alice-aes-1785318084071
EXPECTED: HTTP 200, ENCRYPT_DECRYPT usage, and no key material in response
ACTUAL: HTTP 200, usage = ENCRYPT_DECRYPT

EVIDENCE - Named encryption key creation
{
  "request": {
    "method": "POST",
    "endpoint": "/v1/transit/keys/encryption",
    "requester": "alice.demo.1785318084071@example.com",
    "requested_key_name": "alice-aes-1785318084071"
  },
  "response": {
    "http_status": 200,
    "response_fields": [
      "key_name",
      "key_usage"
    ],
    "key_name": "alice-aes-1785318084071",
    "key_usage": "ENCRYPT_DECRYPT"
  },
  "security_check": {
    "forbidden_response_fields": [
      "key_material",
      "encrypted_key_material",
      "raw_key",
      "secret_key",
      "private_key"
    ],

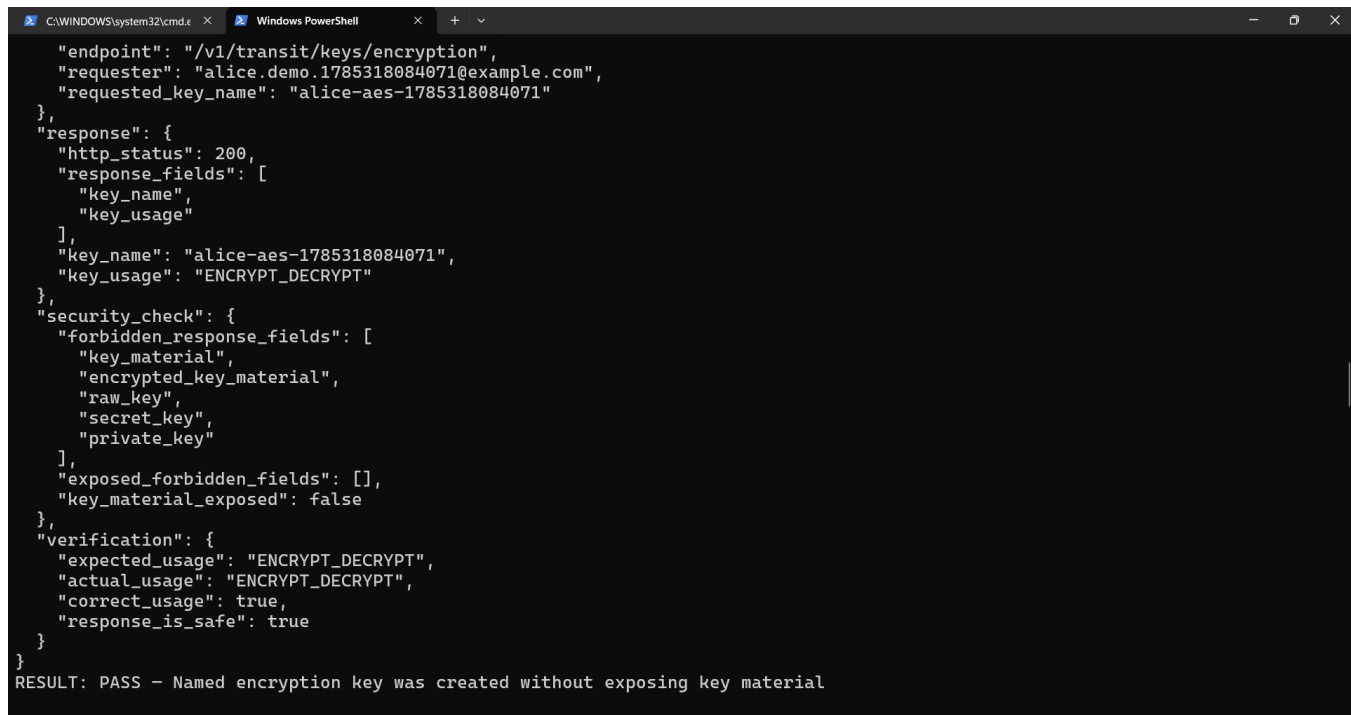
```

Hình 11: Alice tạo named encryption key thành công

Script demo tiếp tục kiểm tra các trường có khả năng làm lộ key material. Kết quả cho thấy:

- `exposed_forbidden_fields` là danh sách rỗng;
- `key_material_exposed = false`;
- `response_is_safe = true`.

Các kết quả trên xác nhận API chỉ trả về metadata cần thiết và không làm lộ key material trong response.



```
"endpoint": "/v1/transit/keys/encryption",
"requester": "alice.demo.1785318084071@example.com",
"requested_key_name": "alice-aes-1785318084071"
},
"response": {
  "http_status": 200,
  "response_fields": [
    "key_name",
    "key_usage"
  ],
  "key_name": "alice-aes-1785318084071",
  "key_usage": "ENCRYPT_DECRYPT"
},
"security_check": {
  "forbidden_response_fields": [
    "key_material",
    "encrypted_key_material",
    "raw_key",
    "secret_key",
    "private_key"
  ],
  "exposed_forbidden_fields": [],
  "key_material_exposed": false
},
"verification": {
  "expected_usage": "ENCRYPT_DECRYPT",
  "actual_usage": "ENCRYPT_DECRYPT",
  "correct_usage": true,
  "response_is_safe": true
}
}
RESULT: PASS - Named encryption key was created without exposing key material
```

Hình 12: Xác minh API không trả về key material trong response

4.6 Chức năng 2.2: Mã hóa và giải mã bằng Transit

4.6.1 Giải thích kỹ thuật

Transit Engine thực hiện các thao tác mật mã thay cho client. Client gửi plaintext đến hệ thống và nhận lại ciphertext nhưng không được truy cập named key dùng để mã hóa.

Khi mã hóa, hệ thống kiểm tra quyền sử dụng named key, giải mã tạm thời key material bằng DEK và sử dụng AES-GCM để mã hóa plaintext. Ciphertext được trả về theo định dạng:

`vault:<key_name>:<payload>`

Khi giải mã, hệ thống phân tích ciphertext để xác định named key tương ứng, kiểm tra quyền truy cập và sử dụng key material để khôi phục plaintext. Key material chỉ tồn tại tạm thời trong bộ nhớ và được zeroize sau khi phép toán hoàn tất.

Hệ thống cũng hỗ trợ xoay vòng named key trong khi vẫn duy trì khả năng giải mã ciphertext được tạo bằng phiên bản khóa cũ. Chức năng này được trình bày trong phần các chức năng nâng cao.

4.6.2 Luồng xử lý

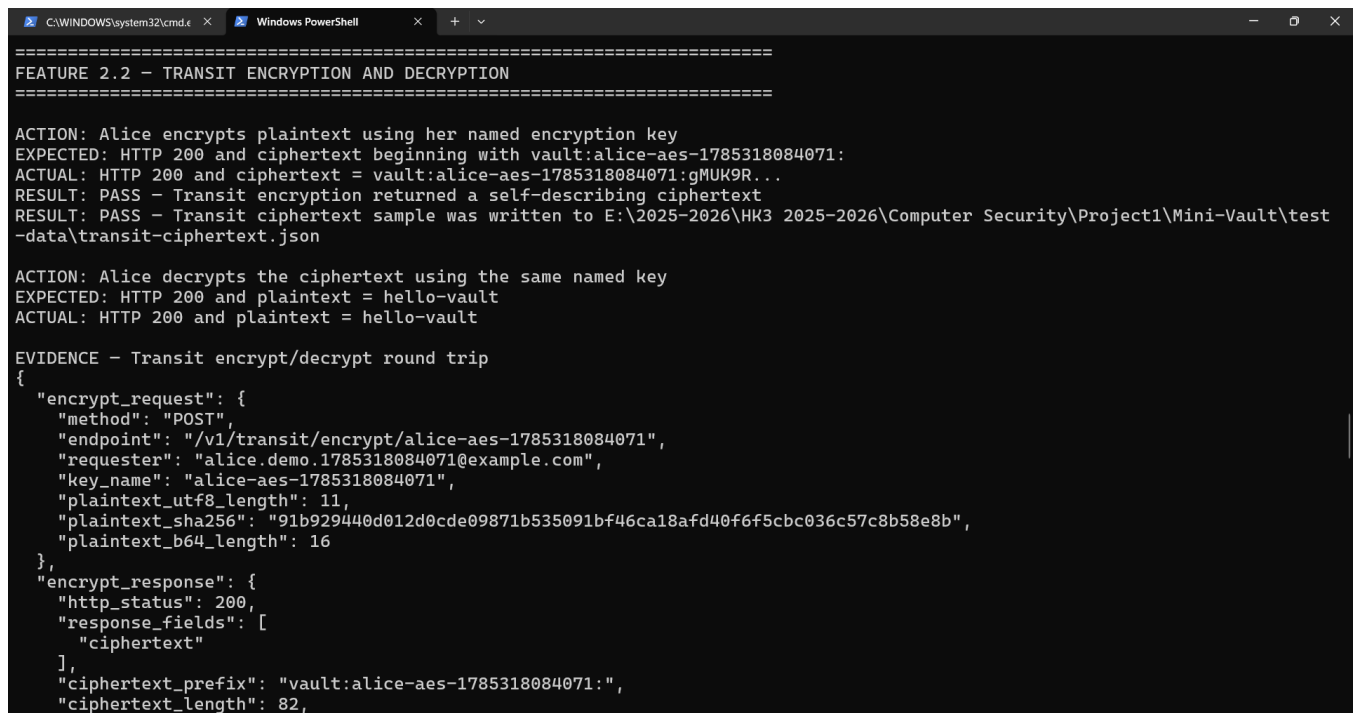
1. Alice gửi plaintext và tên named key đến endpoint mã hóa.

2. Server xác thực session và kiểm tra quyền sử dụng khóa.
3. Key material được giải mã tạm thời bằng DEK.
4. Plaintext được mã hóa bằng AES-GCM.
5. Ciphertext được trả về theo định dạng `vault:<key_name>:<payload>`.
6. Alice gửi ciphertext đến endpoint giải mã.
7. Hệ thống xác định named key và khôi phục plaintext.
8. Plaintext được trả về cho Alice.

4.6.3 Kết quả thực nghiệm

Alice sử dụng named key đã tạo ở Chức năng 2.1 để mã hóa chuỗi `hello-vault`. Request mã hóa trả về HTTP 200 cùng ciphertext có định dạng hợp lệ.

Sau đó, Alice gửi ciphertext đến endpoint giải mã. Request trả về HTTP 200 và plaintext được khôi phục thành `hello-vault`.



```
=====
FEATURE 2.2 - TRANSIT ENCRYPTION AND DECRYPTION
=====

ACTION: Alice encrypts plaintext using her named encryption key
EXPECTED: HTTP 200 and ciphertext beginning with vault:alice-aes-1785318084071:
ACTUAL: HTTP 200 and ciphertext = vault:alice-aes-1785318084071:gMUK9R...
RESULT: PASS - Transit encryption returned a self-describing ciphertext
RESULT: PASS - Transit ciphertext sample was written to E:\2025-2026\HK3 2025-2026\Computer Security\Project1\Mini-Vault\test
-data\transit-ciphertext.json

ACTION: Alice decrypts the ciphertext using the same named key
EXPECTED: HTTP 200 and plaintext = hello-vault
ACTUAL: HTTP 200 and plaintext = hello-vault

EVIDENCE - Transit encrypt/decrypt round trip
{
  "encrypt_request": {
    "method": "POST",
    "endpoint": "/v1/transit/encrypt/alice-aes-1785318084071",
    "requester": "alice.demo.1785318084071@example.com",
    "key_name": "alice-aes-1785318084071",
    "plaintext_utf8_length": 11,
    "plaintext_sha256": "91b929440d012d0cde09871b535091bf46ca18afd40f6f5cbc036c57c8b58e8b",
    "plaintext_b64_length": 16
  },
  "encrypt_response": {
    "http_status": 200,
    "response_fields": [
      "ciphertext"
    ],
    "ciphertext_prefix": "vault:alice-aes-1785318084071:",
    "ciphertext_length": 82,
```

Hình 13: Mã hóa và giải mã dữ liệu thông qua Transit Engine

Script demo đối chiếu dữ liệu trước khi mã hóa và sau khi giải mã. Kết quả cho thấy:

- plaintext_hashes_match = true;
- ciphertext_differs_from_plaintext = true;
- valid_ciphertext_envelope = true.

Các kết quả trên xác nhận ciphertext khác plaintext, định dạng envelope hợp lệ và dữ liệu được khôi phục chính xác sau quá trình giải mã.

```

"plaintext_b64_length": 16
},
"encrypt_response": {
  "http_status": 200,
  "response_fields": [
    "ciphertext"
  ],
  "ciphertext_prefix": "vault:alice-aes-1785318084071:",
  "ciphertext_length": 82,
  "ciphertext_sha256": "3a3f9c2af148a7de32840414eb3c3509c40afd2f5ad9072f70241d945c09b25f",
  "ciphertext_preview": "vault:alice-aes-1785318084071:gMUK9RP9Vc5FJY1YE103vcpCB_CDZ-..."
},
"decrypt_request": {
  "method": "POST",
  "endpoint": "/v1/transit/decrypt",
  "requester": "alice.demo.1785318084071@example.com",
  "ciphertext_sha256": "3a3f9c2af148a7de32840414eb3c3509c40afd2f5ad9072f70241d945c09b25f"
},
"decrypt_response": {
  "http_status": 200,
  "returned_plaintext_sha256": "91b929440d012d0cde09871b535091bf46ca18afd40f6f5cbc036c57c8b58e8b"
},
"verification": {
  "original_plaintext_sha256": "91b929440d012d0cde09871b535091bf46ca18afd40f6f5cbc036c57c8b58e8b",
  "decrypted_plaintext_sha256": "91b929440d012d0cde09871b535091bf46ca18afd40f6f5cbc036c57c8b58e8b",
  "plaintext_hashes_match": true,
  "ciphertext_differs_from_plaintext": true,
  "valid_ciphertext_envelope": true
}
}
RESULT: PASS - Transit encrypt/decrypt round-trip preserved the plaintext

```

Hình 14: Xác minh Transit encrypt/decrypt round trip

4.7 Chức năng 2.3: Kiểm soát truy cập khóa định danh

4.7.1 Giải thích kỹ thuật

Mỗi named key trong bảng transit_keys được liên kết với một chủ sở hữu thông qua trường owner_email. Theo chính sách mặc định, chỉ chủ sở hữu mới được sử dụng khóa.

Việc kiểm tra quyền được thực hiện bởi OwnershipTransitAuthorization. Thành phần này xác định người gửi request, hành động được yêu cầu và named key đích.

Nếu người dùng không phải chủ sở hữu, hệ thống tiếp tục kiểm tra quyền ACL tương ứng. Khi không có quyền phù hợp, request bị từ chối với HTTP 403 và mã lỗi PERMISSION_DENIED.

Kiểm tra này được thực hiện trước khi key material được giải mã và trước khi bất kỳ thao tác mật mã nào được tiến hành. Sự kiện truy cập bị từ chối cũng được ghi nhận bởi `AuditService`.

4.7.2 Luồng xử lý

1. Bob gửi request mã hóa bằng named key của Alice.
2. Server xác thực session token của Bob.
3. Hệ thống xác định Alice là chủ sở hữu của named key.
4. Quyền sở hữu và ACL của Bob được kiểm tra.
5. Do Bob không có quyền phù hợp, request bị từ chối trước khi key material được giải mã.
6. Server trả về HTTP 403 và `PERMISSION_DENIED`.

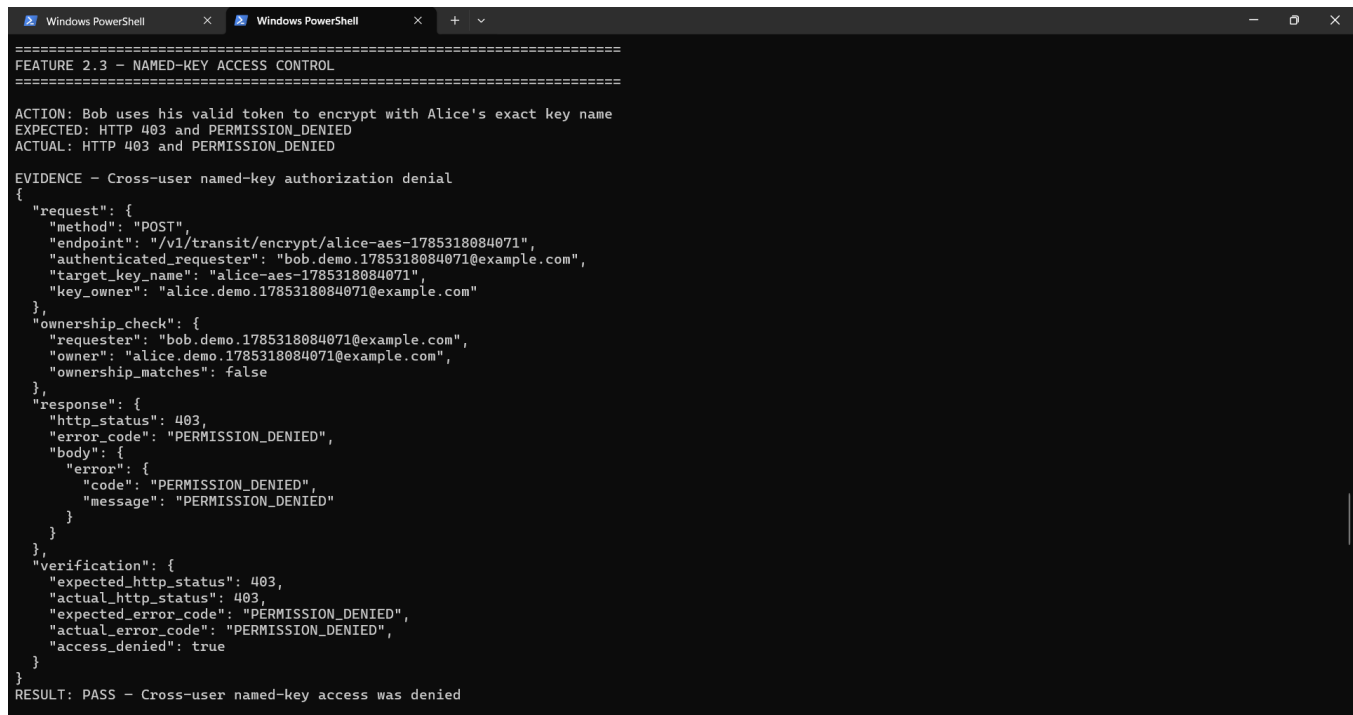
4.7.3 Kết quả thực nghiệm

Bob sử dụng session token hợp lệ để gửi request mã hóa bằng named key do Alice tạo. Tại thời điểm kiểm thử, Alice chưa cấp quyền sử dụng khóa này cho Bob.

Kết quả cho thấy:

- `ownership_matches = false;`
- HTTP status thực tế là 403;
- mã lỗi là `PERMISSION_DENIED`;
- `access_denied = true.`

Kết quả xác nhận Bob đã được xác thực nhưng vẫn không thể sử dụng named key của Alice khi không phải chủ sở hữu và chưa được cấp quyền.



```

=====
FEATURE 2.3 - NAMED-KEY ACCESS CONTROL
=====

ACTION: Bob uses his valid token to encrypt with Alice's exact key name
EXPECTED: HTTP 403 and PERMISSION_DENIED
ACTUAL: HTTP 403 and PERMISSION_DENIED

EVIDENCE - Cross-user named-key authorization denial
{
  "request": {
    "method": "POST",
    "endpoint": "/v1/transit/encrypt/alice-aes-1785318084071",
    "authenticated_requester": "bob.demo.1785318084071@example.com",
    "target_key_name": "alice-aes-1785318084071",
    "key_owner": "alice.demo.1785318084071@example.com"
  },
  "ownership_check": {
    "requester": "bob.demo.1785318084071@example.com",
    "owner": "alice.demo.1785318084071@example.com",
    "ownership_matches": false
  },
  "response": {
    "http_status": 403,
    "error_code": "PERMISSION_DENIED",
    "body": {
      "error": {
        "code": "PERMISSION_DENIED",
        "message": "PERMISSION_DENIED"
      }
    }
  },
  "verification": {
    "expected_http_status": 403,
    "actual_http_status": 403,
    "expected_error_code": "PERMISSION_DENIED",
    "actual_error_code": "PERMISSION_DENIED",
    "access_denied": true
  }
}
RESULT: PASS - Cross-user named-key access was denied

```

Hình 15: Từ chối Bob sử dụng named key của Alice khi chưa được cấp quyền

Trường hợp cấp và thu hồi quyền sử dụng named key bằng ACL được trình bày trong phần các chức năng nâng cao.

4.8 Chức năng 2.4: Ký và xác minh chữ ký

4.8.1 Giải thích kỹ thuật

Chức năng ký và xác minh sử dụng thuật toán ED25519. Named key dành cho chức năng này có mục đích sử dụng `SIGN_VERIFY`.

Khi tạo signing key, hệ thống sinh một cặp khóa gồm private key dùng để ký và public key dùng để xác minh. Private key được mã hóa bằng DEK thông qua AES-GCM trước khi lưu, trong khi public key được lưu để phục vụ quá trình xác minh. Private key không xuất hiện trong response của API.

Khi ký, hệ thống kiểm tra quyền truy cập, giải mã tạm thời private key và tạo chữ ký ED25519 cho message. Sau khi hoàn tất, buffer chứa private key được zeroize.

Khi xác minh, hệ thống sử dụng public key để kiểm tra chữ ký. Message gốc trả về `signature_valid = true`, trong khi message đã bị thay đổi trả về `signature_valid = false`.

Hệ thống còn hỗ trợ cho phép người dùng đã xác thực khác thực hiện thao tác xác minh mà không được cấp quyền ký. Chức năng này được trình bày trong phần các chức năng nâng cao.

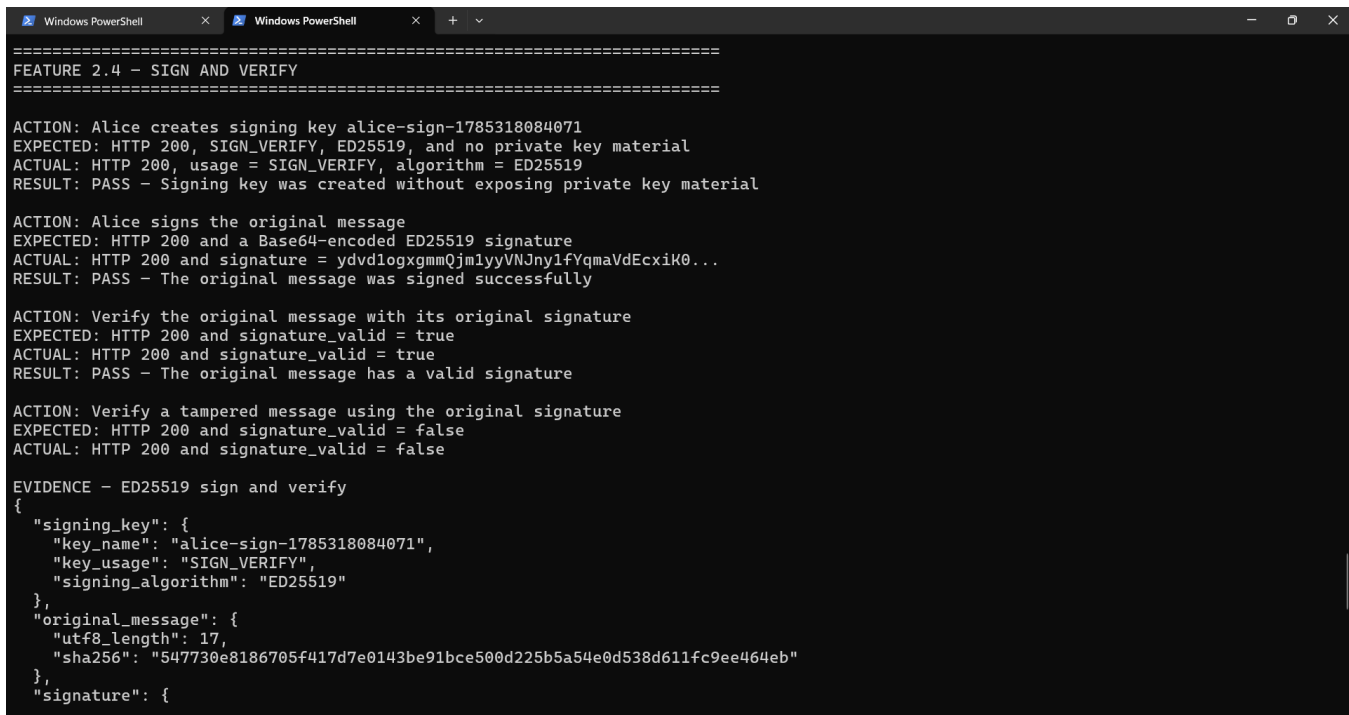
4.8.2 Luồng xử lý

1. Alice tạo một named key loại `SIGN_VERIFY`.
2. Hệ thống sinh cặp khóa ED25519 và mã hóa private key bằng DEK.
3. Alice gửi message đến endpoint ký.
4. Hệ thống tạo chữ ký bằng private key.
5. Message gốc và chữ ký được gửi đến endpoint xác minh.
6. Public key xác minh chữ ký và trả về `signature_valid = true`.
7. Khi message bị thay đổi nhưng vẫn dùng chữ ký cũ, kết quả trả về `signature_valid = false`.

4.8.3 Kết quả thực nghiệm

Alice tạo signing key có mục đích `SIGN_VERIFY` và thuật toán ED25519. Response không chứa private key material.

Sau đó, Alice ký message gốc. Request ký trả về HTTP 200 cùng một chữ ký được mã hóa Base64. Khi xác minh message gốc, hệ thống trả về `signature_valid = true`; khi nội dung message bị thay đổi nhưng vẫn sử dụng chữ ký cũ, kết quả trả về `signature_valid = false`.



```
=====
FEATURE 2.4 - SIGN AND VERIFY
=====

ACTION: Alice creates signing key alice-sign-1785318084071
EXPECTED: HTTP 200, SIGN_VERIFY, ED25519, and no private key material
ACTUAL: HTTP 200, usage = SIGN_VERIFY, algorithm = ED25519
RESULT: PASS - Signing key was created without exposing private key material

ACTION: Alice signs the original message
EXPECTED: HTTP 200 and a Base64-encoded ED25519 signature
ACTUAL: HTTP 200 and signature = ydvdlogxgmmQjm1yyVnJny1fYqmaVdEcxiK0...
RESULT: PASS - The original message was signed successfully

ACTION: Verify the original message with its original signature
EXPECTED: HTTP 200 and signature_valid = true
ACTUAL: HTTP 200 and signature_valid = true
RESULT: PASS - The original message has a valid signature

ACTION: Verify a tampered message using the original signature
EXPECTED: HTTP 200 and signature_valid = false
ACTUAL: HTTP 200 and signature_valid = false

EVIDENCE - ED25519 sign and verify
{
  "signing_key": {
    "key_name": "alice-sign-1785318084071",
    "key_usage": "SIGN_VERIFY",
    "signing_algorithm": "ED25519"
  },
  "original_message": {
    "utf8_length": 17,
    "sha256": "547730e8186705f417d7e0143be91bce500d225b5a54e0d538d611fc9ee464eb"
  },
  "signature": {
```

Hình 16: Tạo signing key, ký message và xác minh chữ ký

Kết quả kiểm tra chi tiết cho thấy:

- `original_verification.signature_valid = true;`
- `tampered_verification.signature_valid = false;`
- `original_message_accepted = true;`
- `tampered_message_rejected = true.`

Các kết quả trên xác nhận chữ ký ED25519 có thể xác minh message gốc và phát hiện khi nội dung message đã bị thay đổi.

```

{
  "signing_key": {
    "key_name": "alice-sign-1785318084071",
    "key_usage": "SIGN_VERIFY",
    "signing_algorithm": "ED25519"
  },
  "original_message": {
    "utf8_length": 17,
    "sha256": "547730e8186705f417d7e0143be91bce500d225b5a54e0d538d611fc9ee464eb"
  },
  "signature": {
    "base64_length": 88,
    "sha256": "e553bddd34097eb73e89db8a05b161fead432bf81aa56e1fc040d4cbfa2e0b1d",
    "preview": "ydvdl0gxgmmQjmlyyVNjny1fYqmaVdEcxiK0jf+3..."
  },
  "original_verification": {
    "http_status": 200,
    "message_sha256": "547730e8186705f417d7e0143be91bce500d225b5a54e0d538d611fc9ee464eb",
    "signature_valid": true
  },
  "tampered_message": {
    "utf8_length": 17,
    "sha256": "8799eb4d1efbb07badfa705d6bd1eaf795f24d02446f3d50f7a62c399a7bf9f3",
    "differs_from_original": true
  },
  "tampered_verification": {
    "http_status": 200,
    "signature_valid": false
  },
  "verification": {
    "original_message_accepted": true,
    "tampered_message_rejected": true
  }
}
RESULT: PASS - The tampered message was detected as invalid

```

Hình 17: Chấp nhận message gốc và từ chối message đã bị thay đổi

5 Kiểm tra dữ liệu mẫu

Theo yêu cầu của đề bài, nhóm cung cấp hai loại dữ liệu mẫu gồm cơ sở dữ liệu chứa KV secret đã được mã hóa và một ciphertext mẫu được tạo bởi Transit Engine. Các file này được lưu trong thư mục `test-data`.

5.1 Cơ sở dữ liệu KV được mã hóa

File `vault-demo.db` là bản sao của cơ sở dữ liệu được tạo trong quá trình chạy demo. Nội dung bảng `kv_entries` được kiểm tra bằng lệnh:

```

1 sqlite3 -line .\test-data\vault-demo.db '
2 "SELECT * FROM kv_entries;"

```

Kết quả cho thấy bản ghi KV chứa các thông tin như KV path, chủ sở hữu, nonce, ciphertext và authentication tag. Dữ liệu JSON gốc không được lưu trong một trường plaintext riêng.

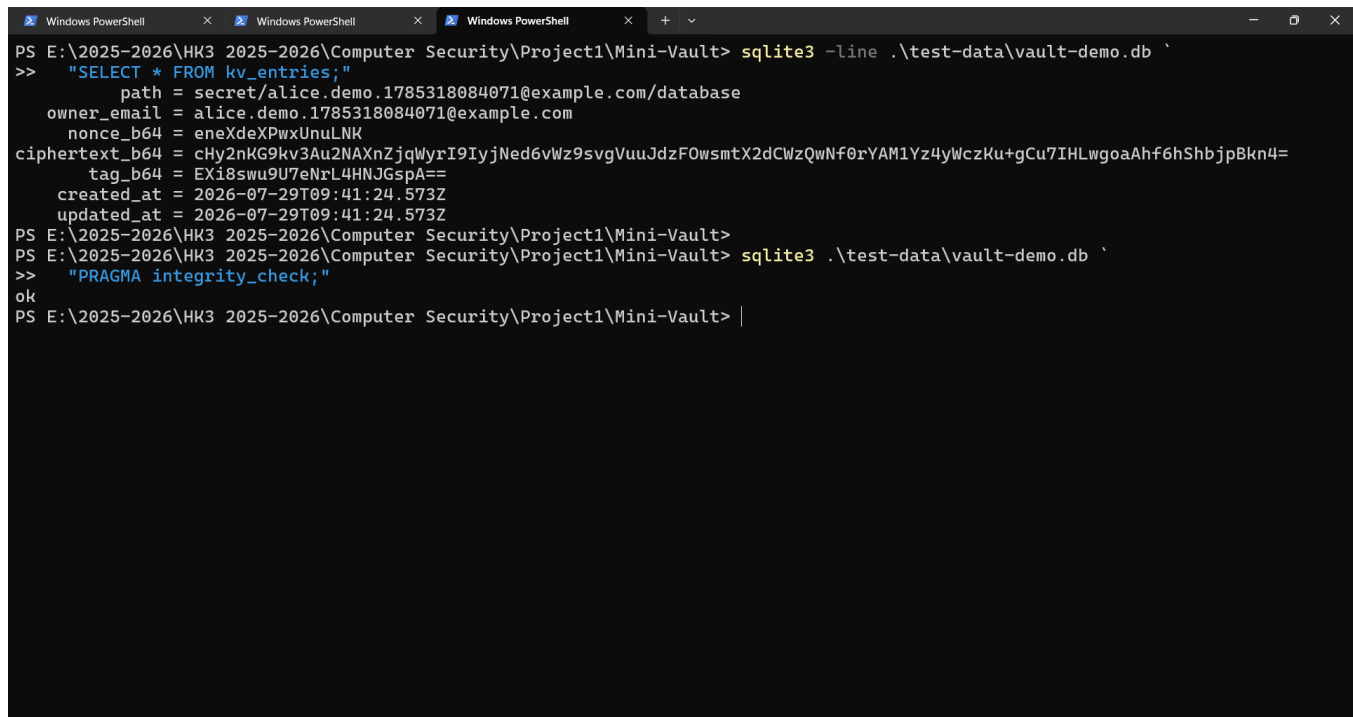
Tính hợp lệ của file SQLite được kiểm tra bằng lệnh:

```

1 sqlite3 .\test-data\vault-demo.db '
2 "PRAGMA integrity_check;"

```

Kết quả trả về ok, xác nhận file cơ sở dữ liệu có cấu trúc hợp lệ và có thể được SQLite đọc thành công.



```

PS E:\2025-2026\HK3 2025-2026\Computer Security\Project1\Mini-Vault> sqlite3 -line .\test-data\vault-demo.db `
>> "SELECT * FROM kv_entries;"
      path = secret/alice.demo.1785318084071@example.com/database
owner_email = alice.demo.1785318084071@example.com
      nonce_b64 = eneXdXPwxUnuLNK
ciphertext_b64 = cHy2nKG9kv3Au2NAXnZjqWyrI9IyjNed6vWz9svgVuuJdzF0wsmtX2dCWzQwNf0rYAM1Yz4yWczKu+gCu7IHLwgoaAhf6hShbjpBkn4=
      tag_b64 = EXi8swu9U7eNrL4HhNJGspA==
      created_at = 2026-07-29T09:41:24.573Z
      updated_at = 2026-07-29T09:41:24.573Z
PS E:\2025-2026\HK3 2025-2026\Computer Security\Project1\Mini-Vault>
PS E:\2025-2026\HK3 2025-2026\Computer Security\Project1\Mini-Vault> sqlite3 .\test-data\vault-demo.db `
>> "PRAGMA integrity_check;"
ok
PS E:\2025-2026\HK3 2025-2026\Computer Security\Project1\Mini-Vault> |

```

Hình 18: Kiểm tra bản ghi KV được mã hóa trong file dữ liệu mẫu

5.2 Transit ciphertext mẫu

File transit-ciphertext.json chứa ciphertext được tạo trong quá trình mã hóa bằng Transit Engine. File lưu tên named key, ciphertext và các thông tin liên quan để phục vụ việc kiểm tra thao tác giải mã.

Định dạng ciphertext được kiểm tra bằng PowerShell:

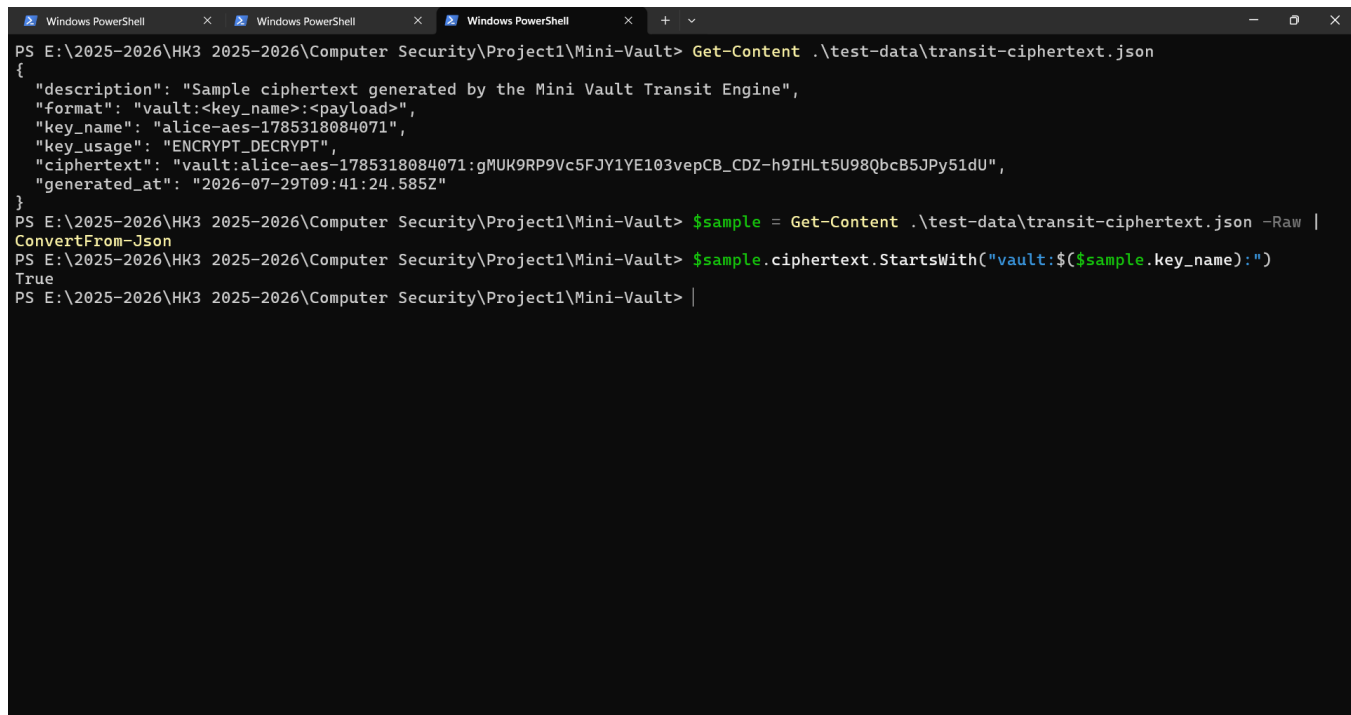
```

1 $sample = Get-Content `
2   .\test-data\transit-ciphertext.json -Raw |
3   ConvertFrom-Json
4
5 $sample.ciphertext.StartsWith(
6   "vault:${$sample.key_name}:"
7 )

```

Kết quả trả về True, xác nhận ciphertext có đúng định dạng:

vault:<key_name>:<payload>



```
PS E:\2025-2026\HK3 2025-2026\Computer Security\Project1\Mini-Vault> Get-Content .\test-data\transit-ciphertext.json
{
  "description": "Sample ciphertext generated by the Mini Vault Transit Engine",
  "format": "vault:<key_name>:<payload>",
  "key_name": "alice-aes-1785318084071",
  "key_usage": "ENCRYPT_DECRYPT",
  "ciphertext": "vault:alice-aes-1785318084071:gMUK9RP9Vc5FJY1YE103vcpCB_CDZ-h9IHLt5U98QbcB5JPy51dU",
  "generated_at": "2026-07-29T09:41:24.585Z"
}
PS E:\2025-2026\HK3 2025-2026\Computer Security\Project1\Mini-Vault> $sample = Get-Content .\test-data\transit-ciphertext.json -Raw |
ConvertFrom-Json
PS E:\2025-2026\HK3 2025-2026\Computer Security\Project1\Mini-Vault> $sample.ciphertext.StartsWith("vault:$($sample.key_name):")
True
PS E:\2025-2026\HK3 2025-2026\Computer Security\Project1\Mini-Vault> |
```

Hình 19: Kiểm tra file Transit ciphertext mẫu

Hai file dữ liệu mẫu trên xác nhận hệ thống có thể cung cấp bản ghi KV được lưu dưới dạng mã hóa và ciphertext Transit đúng cấu trúc theo yêu cầu của đề bài.

6 Các chức năng nâng cao

Ngoài các chức năng bắt buộc, nhóm đã triển khai thêm bảy chức năng nâng cao nhằm tăng cường khả năng xác thực, phân quyền, quản lý phiên bản và bảo vệ tính toàn vẹn của hệ thống. Các chức năng được kiểm tra bằng script `scripts/demo-advanced.ts` thông qua lệnh:

```
npm run demo:advanced
```

Mỗi trường hợp kiểm thử hiển thị hành động thực hiện, kết quả mong đợi, evidence thực tế và kết luận PASS hoặc FAIL.

6.1 Xác thực đa yếu tố bằng TOTP

6.1.1 Giải thích kỹ thuật

Hệ thống hỗ trợ xác thực đa yếu tố bằng *Time-based One-Time Password* (TOTP). Sau khi MFA được kích hoạt, đăng nhập bằng email và passphrase không trả về session token trực tiếp mà tạo một MFA challenge để người dùng tiếp tục xác minh mã TOTP.

6.1.2 Kết quả thực nghiệm

MFA được thiết lập thành công khi Vault vẫn ở trạng thái LOCKED. Sau khi kích hoạt, lần đăng nhập tiếp theo trả về các trường `mfa_required`, `mfa_token` và `email`, trong khi `session_issued_before_totp` có giá trị `false`. Kết quả xác nhận hệ thống không cấp session trước khi hoàn tất bước xác thực MFA.

```
=== ADVANCED 1 - MFA/TOTP AUTHENTICATION ===  
ACTION: Enable TOTP MFA for Alice while the Vault remains locked  
EXPECTED: MFA setup and enable succeed; Vault remains LOCKED; password login no longer returns a session directly  
EVIDENCE: {  
  vault_status_before: 'LOCKED',  
  mfa_setup_returned_secret: true,  
  mfa_enable_completed: true,  
  session_issued_before_totp: false,  
  login_response_fields: [ 'mfa_required', 'mfa_token', 'email' ],  
  vault_status_after: 'LOCKED'  
}  
RESULT: PASS - MFA was enabled while the Vault remained locked, and password login did not directly issue a session
```

Hình 20: Kích hoạt TOTP và yêu cầu bước xác thực MFA khi đăng nhập

6.2 Audit log chống chỉnh sửa

6.2.1 Giải thích kỹ thuật

Các bản ghi audit được liên kết thành một chuỗi hash. Hash của mỗi entry phụ thuộc vào nội dung sự kiện và hash của entry trước đó. Vì vậy, việc thay đổi một bản ghi cũ có thể làm chuỗi không còn hợp lệ.

6.2.2 Kết quả thực nghiệm

Sau khi Vault được mở khóa, script kiểm tra toàn bộ chuỗi audit hiện có. Kết quả `chain_valid = true` và `entries_checked = 5`, xác nhận năm audit entry đã được kiểm tra và chuỗi hash hiện tại hợp lệ.

```
=== ADVANCED 2 - TAMPER-EVIDENT AUDIT LOG ===  
ACTION: Unlock the Vault and verify the audit hash chain  
EXPECTED: Vault becomes UNLOCKED and all existing audit entries form a valid hash chain  
EVIDENCE: { vault_status: 'UNLOCKED', chain_valid: true, entries_checked: 5 }  
RESULT: PASS - The Vault was unlocked and the audit hash chain was valid
```

Hình 21: Xác minh tính hợp lệ của chuỗi hash trong audit log

6.3 Quản lý phiên bản KV

6.3.1 Giải thích kỹ thuật

KV versioning cho phép lưu lại lịch sử của một secret sau mỗi lần cập nhật. Người dùng có thể liệt kê các phiên bản hiện có và đọc lại một phiên bản cụ thể thay vì chỉ truy xuất dữ liệu mới nhất.

6.3.2 Kết quả thực nghiệm

Alice ghi lần lượt hai giá trị {n: 1} và {n: 2} vào cùng một KV path. Hệ thống liệt kê được hai phiên bản [1, 2], trong đó phiên bản 1 vẫn trả về {n: 1}, còn phiên bản hiện tại là 2 với dữ liệu {n: 2}.

```
=== ADVANCED 3 - KV VERSIONING ===  
ACTION: Write two versions of a secret and read Version 1  
EXPECTED: Versions 1 and 2 are listed; Version 1 remains readable after Version 2 is written  
EVIDENCE: {  
  path: 'secret/alice@example.com/demo',  
  listed_versions: [ 1, 2 ],  
  version_1_data: { n: 1 },  
  current_version: 2,  
  current_data: { n: 2 }  
}  
RESULT: PASS - Two KV versions were stored and Version 1 remained readable
```

Hình 22: Lưu hai phiên bản và đọc lại phiên bản cũ của KV secret

6.4 Xoay vòng khóa Transit

6.4.1 Giải thích kỹ thuật

Transit key rotation cho phép sinh một phiên bản key material mới cho named key. Phiên bản mới được sử dụng cho các thao tác mã hóa tiếp theo, trong khi các phiên bản cũ vẫn được duy trì để giải mã ciphertext đã tạo trước đó.

6.4.2 Kết quả thực nghiệm

Alice mã hóa plaintext `rotate-me`, sau đó xoay vòng named key sang phiên bản 2. Ciphertext cũ vẫn được giải mã thành công với `recovered_plaintext = rotate-me` và `old_ciphertext_decrypts = true`.

```
=== ADVANCED 4 - TRANSIT KEY ROTATION ===
ACTION: Encrypt plaintext, rotate the named key, then decrypt the old ciphertext
EXPECTED: The ciphertext created before rotation still decrypts to "rotate-me"
EVIDENCE: {
  key_before_rotation: { key_name: 'k', key_usage: 'ENCRYPT_DECRYPT' },
  key_after_rotation: { key_name: 'k', current_version: 2 },
  old_ciphertext_prefix: 'vault:k:-bvz2E38e-9yM6MgEMXWnG',
  recovered_plaintext: 'rotate-me',
  old_ciphertext_decrypts: true
}
RESULT: PASS - The old ciphertext remained decryptable after Transit key rotation
```

Hình 23: Giải mã ciphertext cũ sau khi xoay vòng Transit key

6.5 Phân quyền bằng ACL

6.5.1 Giải thích kỹ thuật

ACL cho phép chủ sở hữu cấp quyền truy cập một tài nguyên cụ thể cho người dùng khác. Quyền này được kiểm tra cùng với ownership và có thể được thu hồi khi không còn cần thiết.

6.5.2 Kết quả thực nghiệm

Trước khi được cấp quyền, Bob bị từ chối khi đọc KV secret của Alice. Sau khi Alice cấp permission `read`, Bob đọc được dữ liệu `{n: 2}`. Khi quyền bị thu hồi, Bob tiếp tục nhận lỗi `PERMISSION_DENIED`. Kết quả xác nhận đầy đủ quá trình cấp và thu hồi quyền ACL.


```
=== ADVANCED 5 - ACL GRANT AND REVOKE ===
ACTION: Test Bob before grant, grant read permission, then revoke the permission
EXPECTED: Bob is denied before grant, can read after grant, and is denied again after revoke
EVIDENCE: {
  denied_before_grant: true,
  error_before_grant: 'PERMISSION_DENIED',
  read_after_grant: true,
  data_after_grant: { n: 2 },
  revoke_completed: true,
  denied_after_revoke: true,
  error_after_revoke: 'PERMISSION_DENIED'
}
RESULT: PASS - ACL grant allowed Bob to read, and revoke removed the permission
```

Hình 24: Cấp và thu hồi quyền đọc KV secret thông qua ACL

6.6 Chia sẻ quyền xác minh chữ ký

6.6.1 Giải thích kỹ thuật

Signing key có thể được cấu hình để cho phép người dùng đã xác thực khác thực hiện thao tác xác minh chữ ký. Cơ chế này chỉ chia sẻ quyền `verify`; private key vẫn được bảo vệ và không thể được sử dụng để ký bởi người dùng khác.

6.6.2 Kết quả thực nghiệm

Alice ký một message bằng signing key của mình. Bob xác minh chữ ký thành công với `signature_valid = true`, nhưng thao tác ký bằng cùng named key bị từ chối với `PERMISSION_DENIED`. Điều này xác nhận quyền xác minh không đồng nghĩa với quyền ký.

```
=== ADVANCED 6 - SHARED SIGNATURE VERIFICATION ===
ACTION: Alice signs a message; Bob verifies it and attempts to sign with Alice's key
EXPECTED: Bob can verify the signature but cannot sign with Alice's private key
EVIDENCE: {
  verifier: 'bob@example.com',
  signature_valid: true,
  signing_algorithm: 'ED25519',
  bob_sign_denied: true,
  bob_sign_error: 'PERMISSION_DENIED'
}
RESULT: PASS - Bob could verify Alice's signature but could not sign with Alice's key
```

Hình 25: Bob được phép xác minh nhưng không được ký bằng khóa của Alice

6.7 Shamir's Secret Sharing

6.7.1 Giải thích kỹ thuật

Shamir's Secret Sharing chia bí mật mở khóa Vault thành N share và yêu cầu ít nhất K share hợp lệ để khôi phục. Trong thử nghiệm, hệ thống sử dụng cấu hình 3-trên-5.

6.7.2 Kết quả thực nghiệm

Khi chỉ cung cấp hai share, thao tác mở khóa bị từ chối và Vault vẫn ở trạng thái LOCKED. Khi cung cấp ba trong tổng số năm share, Vault chuyển sang trạng thái UNLOCKED. Kết quả xác nhận ngưỡng 3-trên-5 được thực thi đúng.

```
=== ADVANCED 7 - SHAMIR SECRET SHARING ===  
ACTION: Attempt to unlock with 2 of 5 shares, then unlock with the required 3 shares  
EXPECTED: Two shares are insufficient; three shares unlock the Vault  
EVIDENCE: {  
  total_shares: 5,  
  required_shares: 3,  
  unlock_with_2_shares_denied: true,  
  status_after_2_shares: 'LOCKED',  
  status_after_3_shares: 'UNLOCKED'  
}  
RESULT: PASS - The Vault rejected 2 shares and unlocked successfully with 3 of 5 shares
```

Hình 26: Kiểm tra ngưỡng 3-trên-5 của Shamir's Secret Sharing